

使用 Java - SpringBoot 進行 InnerLoop 開發

來源：<https://codelabs.developers.google.com/innerloop-dev-java-springboot?hl=zh-tw>

步驟數：6

瞭解專為在容器化環境中開發 Java 應用程式的軟體工程師所設計的功能和功能，簡化開發工作流程。

目錄

1. [1. 總覽](#)
2. [2. 設定和需求](#)
3. [3. 建立新的 Java 啟動應用程式](#)
4. [4. 逐步完成開發程序](#)
5. [5. 開發簡易的 CRUD REST 服務](#)
6. [6. 清除所用資源](#)

步驟 1 · 約 0 分鐘

1. 總覽

本實驗室將展示各項功能，協助軟體工程師在容器化環境中開發 Java 應用程式時，簡化開發工作流程。一般容器開發作業需要使用者瞭解容器的詳細資料和容器建構程序。此外，開發人員通常必須中斷流程，離開 IDE 在遠端環境中測試及偵錯應用程式。有了本教學課程中提及的工具和技術，開發人員就能在 IDE 中有效處理容器化應用程式。

學習目標

在本實驗室中，您將瞭解如何在 GCP 中使用容器進行開發，包括：

- 設定和需求
 - 建立新的 Java 啟動應用程式
 - 逐步完成開發程序
 - 開發簡易的 CRUD REST 服務
 - 清除所用資源
-

步驟 2 · 約 2 分鐘

2. 設定和需求

自修實驗室環境設定

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。

The screenshot shows the Google Cloud Platform interface. At the top, there's a blue header with the Google Cloud Platform logo and a 'Select a project' button circled in blue. Below this is a 'Select a project' section with a search bar. Further down is the 'New Project' section. It contains a 'Project name' field with the text 'my-kubernetes-codelab' and a question mark icon, which is circled in blue. Below the name field is the 'Project ID' field, which contains the text 'my-kubernetes-codelab. It cannot be changed later.' and an 'EDIT' link. Below the ID field is the 'Location' field with the text 'No organization' and a 'BROWSE' link. At the bottom of the 'New Project' section are 'CREATE' and 'CANCEL' buttons.

- **專案名稱**是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串，您隨時可以更新。
- **專案 ID** 在所有 Google Cloud 專案中不得重複，且設定後即無法變更。Cloud 控制台會自動產生專屬字串，通常您不需要在意該字串為何。在大多數程式碼研究室中，您需要參照專案 ID (通常會標示為 `PROJECT_ID`)，因此如果您不喜歡該字串，可以產生另一個隨機字串，或是嘗試使用自己的字串，看看是否可用。專案建立後，系統就會「凍結」該值。
- 還有第三個值，也就是部分 API 使用的「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中不得重複，選定後即為專屬 ID，其他使用者無法使用。您是該 ID 的唯一使用者。專案刪除後，該 ID 就無法再使用。

注意：如果您使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果您使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

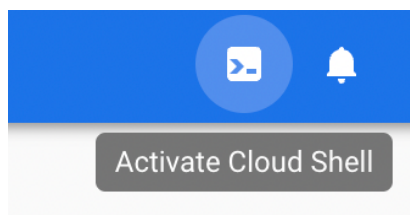
2. 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成本程式碼研究室的費用應該不高，甚至完全免費。如要停用資源，避免在本教學課程結束後繼續產生帳單費用，請按照程式碼研究室結尾的「清除」操作說明

操作。Google Cloud 新使用者可參加[價值\\$300 美元的免費試用計畫](#)。

啟動 Cloud Shell 編輯器

本實驗室專為 Google Cloud Shell 編輯器設計及測試，如要存取編輯器，請按照下列步驟操作：

1. 前往 <https://console.cloud.google.com> 存取 Google 專案。
2. 按一下右上角的 Cloud Shell 編輯器圖示



3. 視窗底部會開啟新窗格
4. 按一下「開啟編輯器」按鈕



5. 編輯器會開啟，右側顯示檔案總管，中央區域則顯示編輯器
6. 畫面底部也應顯示終端機窗格
7. 如果終端機尚未開啟，請使用 `ctrl+`` 鍵組合開啟新的終端機視窗

設定 gcloud

在 Cloud Shell 中，設定專案 ID 和要部署應用程式的區域。分別儲存為 `PROJECT_ID` 和 `REGION` 變數。

```
export PROJECT_ID=$(gcloud config get-value project)
export PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --format='value(projectNumber)')
```

取得原始碼

本實驗室的原始碼位於 GitHub 的 GoogleCloudPlatform 容器開發人員研討會。使用下列指令複製，然後切換至該目錄。

```
git clone https://github.com/GoogleCloudPlatform/container-developer-workshop.git
cd container-developer-workshop/labs/spring-boot
```

佈建本實驗室使用的基礎架構

在本實驗室中，您將程式碼部署至 GKE，並存取儲存在 Cloud SQL 資料庫中的資料。下方的設定指令碼會為您準備好基礎架構。佈建程序需要 10 分鐘以上。設定程序處理期間，你可以繼續進行後續步驟。

```
./setup.sh
```

3. 建立新的 Java 啟動應用程式

在本節中，您將從頭開始建立新的 Java Spring Boot 應用程式，並使用 spring.io 提供的範例應用程式。

複製範例應用程式

1. 建立入門應用程式

```
curl https://start.spring.io/starter.zip -d dependencies=web -d type=maven-project -d javaVersion=11 -d packageName=com.example.springboot -o sample-app.zip
```

2. 解壓縮應用程式

```
unzip sample-app.zip -d sample-app
```

3. 切換至 sample-app 目錄，並在 Cloud Shell IDE 工作區中開啟該資料夾

```
cd sample-app && cloudshell workspace .
```

新增 spring-boot-devtools 和 Jib

如要啟用 Spring Boot DevTools，請在編輯器的檔案總管中尋找並開啟 `pom.xml`。接著，在說明行 (讀取 `<description>Demo project for Spring Boot</description>`) 後方貼上下列程式碼

1. 在 `pom.xml` 中新增 `spring-boot-devtools`

開啟專案根目錄中的 `pom.xml`。在 `Description` 項目後方新增下列設定。

`pom.xml`

```
<!-- Spring profiles-->
<profiles>
  <profile>
    <id>sync</id>
    <dependencies>
      <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-devtools</artifactId>
      </dependency>
    </dependencies>
  </profile>
</profiles>
```

2. 在 `pom.xml` 中啟用 `jib-maven-plugin`

Jib 是 Google 推出的開放原始碼 Java 容器化工具，可讓 Java 開發人員使用熟悉的 Java 工具建構容器。Jib 是一種快速又簡單的容器映像檔建構工具，可處理將應用程式封裝為容器映像檔的所有步驟。您不需要編寫 Dockerfile 或安裝 Docker，而且這項功能直接整合至 Maven 和 Gradle。

在 pom.xml 檔案中向下捲動，更新 `Build` 區段，加入 Jib 外掛程式。完成後，建構部分應如下所示。

pom.xml

```
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
    <!-- Jib Plugin-->
    <plugin>
      <groupId>com.google.cloud.tools</groupId>
      <artifactId>jib-maven-plugin</artifactId>
      <version>3.2.0</version>
    </plugin>
    <!-- Maven Resources Plugin-->
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-resources-plugin</artifactId>
      <version>3.1.0</version>
    </plugin>
  </plugins>
</build>
```

如果系統提示您變更建構檔案，請選擇 `Always`。

 A build file was modified. Do you want to synchronize the Java classpath/configuration?

Never

Now

Always

產生資訊清單

Scaffold 提供整合式工具，可簡化容器開發作業。在這個步驟中，您將初始化 Scaffold，系統會自動建立基本 Kubernetes YAML 檔案。這個程序會嘗試找出含有容器映像檔定義的目錄 (例如 Dockerfile)，然後為每個目錄建立部署和服務資訊清單。

執行下列指令，開始進行程序。

1. 在終端機中執行下列指令

```
scaffold init --generate-manifests
```

2. 出現提示時：

- 使用箭頭將游標移至 `Jib Maven Plugin`
- 按下空格鍵選取選項。
- 按下 Enter 鍵即可繼續

3. 輸入 **8080** 做為通訊埠

4. 輸入 **y** 儲存設定

兩個檔案會新增至工作區，分別是 `scaffold.yaml` 和 `deployment.yaml`。

更新應用程式名稱

設定中包含的預設值目前與應用程式名稱不符。更新檔案，參照應用程式名稱，而非預設值。

1. 變更 Skaffold 設定中的項目

- 開啟「`scaffold.yaml`」
- 選取目前設為 `pom-xml-image` 的圖片名稱
- 按一下滑鼠右鍵，然後選擇「變更所有出現位置」
- 輸入新名稱，如 `demo-app`

2. 變更 Kubernetes 設定中的項目

- 開啟 `deployment.yaml` 檔案
- 選取目前設為 `pom-xml-image` 的圖片名稱
- 按一下滑鼠右鍵，然後選擇「變更所有出現位置」
- 輸入新名稱，如 `demo-app`

啟用熱同步

為獲得最佳熱重載體驗，您將使用 Jib 提供的同步功能。在本步驟中，您將設定 Skaffold，在建構程序中使用這項功能。

請注意，您在 skaffold 設定中設定的「sync」設定檔，會運用您在上一個步驟中設定的 Spring「sync」設定檔，並啟用對 spring-dev-tools 的支援。

1. 更新 Skaffold 設定

在 `scaffold.yaml` 檔案中，將檔案的整個建構區段替換為下列規格。請勿變更檔案的其他部分。

skaffold.yaml

```
build:
  artifacts:
  - image: demo-app
  jib:
    project: com.example:demo
    type: maven
    args:
    - --no-transfer-progress
    - -Psync
    fromImage: gcr.io/distroless/java:debug
  sync:
    auto: true
```

新增預設路由

在 `/src/main/java/com/example/springboot/` 建立名為 `HelloController.java` 的檔案。

將下列內容貼到檔案中，建立預設的 HTTP 路由

HelloController.java

```
package com.example.springboot;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.beans.factory.annotation.Value;

@RestController
public class HelloController {

    @Value("${target:local}")
    String target;

    @GetMapping("/")
    public String hello()
    {
        return String.format("Hello from your %s environment!", target);
    }
}
```

4. 逐步完成開發程序

在本節中，您將使用 Cloud Code 外掛程式逐步完成幾個步驟，瞭解基本程序，並驗證入門應用程式的設定。

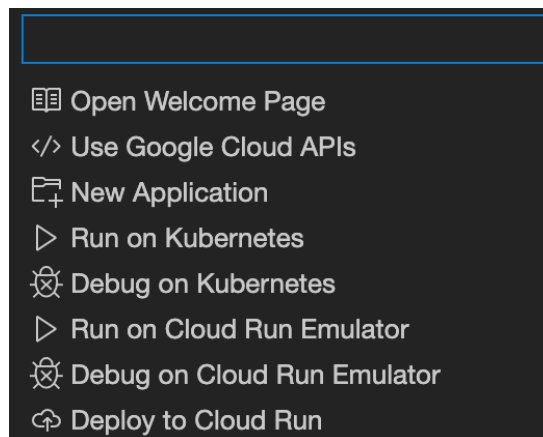
Cloud Code 整合了 Skaffold，可簡化開發程序。在後續步驟中部署至 GKE 時，Cloud Code 和 Skaffold 會自動建構容器映像檔、將其推送至 Container Registry，然後將應用程式部署至 GKE。這項作業會在幕後進行，將詳細資料從開發人員流程中抽象化。此外，Cloud Code 也為以容器為基礎的開發作業提供傳統的偵錯和熱同步功能，進一步提升開發流程。

部署到 Kubernetes

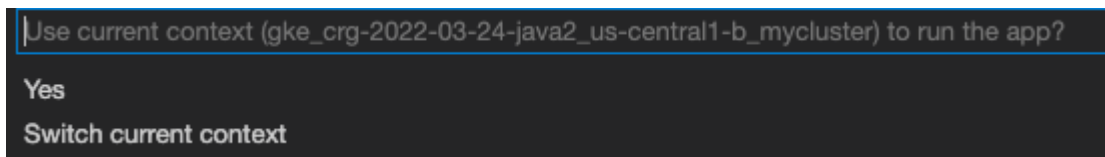
1. 在 Cloud Shell 編輯器底部的窗格中，選取 Cloud Code 

 Cloud Code

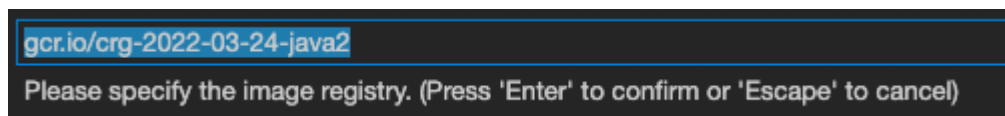
2. 在頂端顯示的面板中，選取「在 Kubernetes 上偵錯」。如果系統顯示提示，請選取「Yes」使用目前的 Kubernetes 環境。



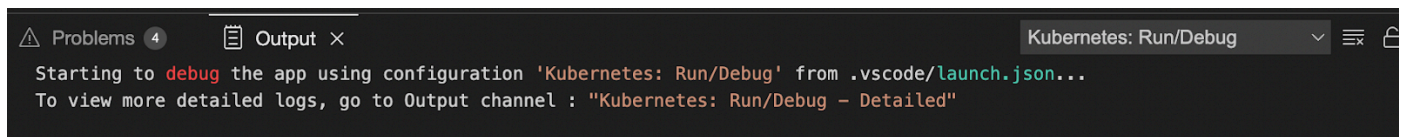
3. 首次執行指令時，畫面頂端會顯示提示，詢問您是否要使用目前的 Kubernetes 內容，請選取「Yes」接受並使用目前的內容。



4. 接著系統會顯示提示，詢問要使用哪個容器登錄服務。按下 Enter 鍵接受預設值



5. 選取下方窗格中的「輸出」分頁標籤，即可查看進度 and 通知



6. 在右側的管道下拉式選單中選取「Kubernetes: Run/Debug - Detailed」，即可查看其他詳細資料和容器的即時記錄檔串流

7. 從下拉式選單選取「Kubernetes: Run/Debug」，即可返回簡化檢視畫面
8. 建構和測試完成後，「輸出」分頁會顯示 `Resource deployment/demo-app status completed successfully`，並列出網址：「Forwarded URL from service demo-app: `http://localhost: 8080`」（從服務 demo-app 轉送的網址：`http://localhost:8080`）
9. 在 Cloud Code 終端機中，將游標懸停在輸出內容中的網址 (`http://localhost:8080`) 上，然後在顯示的工具提示中選取「Open Web Preview」（開啟網頁預覽）。

回覆內容如下：

```
Hello from your local environment!
```

運用中斷點

1. 開啟位於 `/src/main/java/com/example/springboot/HelloController.java` 的 `HelloController.java` 應用程式
2. 找出根路徑的傳回敘述，內容為 `return String.format("Hello from your %s environment!", target);`
3. 點選行號左側的空白處，在該行新增中斷點。系統會顯示紅色指標，表示已設定中斷點
4. 重新載入瀏覽器，並注意偵錯工具會在該中斷點停止程序，讓您調查在 GKE 中遠端執行的應用程式的變數和狀態
5. 按一下變數部分，直到找到「目標」變數為止。
6. 觀察目前值為「local」
7. 按兩下變數名稱「target」，然後在彈出式視窗中將值變更為其他值，例如「Cloud」
8. 按一下偵錯控制面板中的「繼續」按鈕
9. 在瀏覽器中查看回應，現在應該會顯示您剛輸入的更新值。

熱重載

1. 將陳述式變更為傳回其他值，例如「Hello from %s Code」
2. 檔案會自動儲存並同步到 GKE 中的遠端容器
3. 重新整理瀏覽器即可查看更新後的結果。
4. 按一下偵錯工具列中的紅色方塊



，停止偵錯工作階段。

5. 開發簡易的 CRUD REST 服務

此時，您的應用程式已完全設定為容器化開發，且您已透過 Cloud Code 逐步瞭解基本開發工作流程。在接下來的章節中，您將新增 REST 服務端點，連線至 Google Cloud 中的代管資料庫，藉此練習所學內容。

設定依附元件

應用程式程式碼會使用資料庫保存 REST 服務資料。在 `pom.xml` 中新增下列內容，確保依附元件可用

1. 開啟 `pom.xml` 檔案，並在設定的依附元件區段中新增下列內容

`pom.xml`

```
<!-- Database dependencies-->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>org.flywaydb</groupId>
  <artifactId>flyway-core</artifactId>
</dependency>
```

編寫 REST 服務的程式碼

`Quote.java`

在 `/src/main/java/com/example/springboot/` 中建立名為 `Quote.java` 的檔案，然後複製下列程式碼。這會定義應用程式中使用的 Quote 物件的 Entity 模型。

```
package com.example.springboot;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.Table;
import java.util.Objects;

@Entity
@Table(name = "quotes")
public class Quote
{
    @Id
    @Column(name = "id")
    private Integer id;

    @Column(name="quote")
    private String quote;

    @Column(name="author")
    private String author;

    public Integer getId() {
        return id;
    }

    public void setId(Integer id) {
        this.id = id;
    }

    public String getQuote() {
        return quote;
    }

    public void setQuote(String quote) {
        this.quote = quote;
    }

    public String getAuthor() {
        return author;
    }

    public void setAuthor(String author) {
        this.author = author;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) {
            return true;
        }
        if (o == null || getClass() != o.getClass()) {
```

```

        return false;
    }

    Quote quote1 = (Quote) o;
    return Objects.equals(id, quote1.id) &&
        Objects.equals(quote, quote1.quote) &&
        Objects.equals(author, quote1.author);
}

@Override
public int hashCode() {
    return Objects.hash(id, quote, author);
}
}

```

QuoteRepository.java

在 `src/main/java/com/example/springboot` 建立名為 `QuoteRepository.java` 的檔案，然後複製下列程式碼

```

package com.example.springboot;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;

public interface QuoteRepository extends JpaRepository<Quote,Integer> {

    @Query( nativeQuery = true, value =
        "SELECT id,quote,author FROM quotes ORDER BY RANDOM() LIMIT 1")
    Quote findRandomQuote();
}

```

這段程式碼會使用 JPA 持續保存資料。這個類別會擴充 Spring `JpaRepository` 介面，並允許建立自訂程式碼。在您新增的程式碼中，加入 `findRandomQuote` 自訂方法。

QuoteController.java

如要公開服務的端點，`QuoteController` 類別會提供這項功能。

在 `src/main/java/com/example/springboot` 建立名為 `QuoteController.java` 的檔案，並複製下列內容

```

package com.example.springboot;

import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

import org.springframework.dao.EmptyResultDataAccessException;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class QuoteController {

    private final QuoteRepository quoteRepository;

    public QuoteController(QuoteRepository quoteRepository) {
        this.quoteRepository = quoteRepository;
    }

    @GetMapping("/random-quote")
    public Quote randomQuote()
    {
        return quoteRepository.findRandomQuote();
    }

    @GetMapping("/quotes")
    public ResponseEntity<List<Quote>> allQuotes()
    {
        try {
            List<Quote> quotes = new ArrayList<Quote>();

            quoteRepository.findAll().forEach(quotes::add);

            if (quotes.size()==0 || quotes.isEmpty())
                return new ResponseEntity<List<Quote>>(HttpStatus.NO_CONTENT);

            return new ResponseEntity<List<Quote>>(quotes, HttpStatus.OK);
        } catch (Exception e) {
            System.out.println(e.getMessage());
            return new ResponseEntity<List<Quote>>(HttpStatus.INTERNAL_SERVER_ERROR);
        }
    }

    @PostMapping("/quotes")
    public ResponseEntity<Quote> createQuote(@RequestBody Quote quote) {

```

```

    try {
        Quote saved = quoteRepository.save(quote);
        return new ResponseEntity<Quote>(saved, HttpStatus.CREATED);
    } catch (Exception e) {
        System.out.println(e.getMessage());
        return new ResponseEntity<Quote>(HttpStatus.INTERNAL_SERVER_ERROR);
    }
}

@PutMapping("/quotes/{id}")
public ResponseEntity<Quote> updateQuote(@PathVariable("id") Integer id, @RequestBody
Quote quote) {
    try {
        Optional<Quote> existingQuote = quoteRepository.findById(id);

        if(existingQuote.isPresent()){
            Quote updatedQuote = existingQuote.get();
            updatedQuote.setAuthor(quote.getAuthor());
            updatedQuote.setQuote(quote.getQuote());

            return new ResponseEntity<Quote>(updatedQuote, HttpStatus.OK);
        } else {
            return new ResponseEntity<Quote>(HttpStatus.NOT_FOUND);
        }
    } catch (Exception e) {
        System.out.println(e.getMessage());
        return new ResponseEntity<Quote>(HttpStatus.INTERNAL_SERVER_ERROR);
    }
}

>DeleteMapping("/quotes/{id}")
public ResponseEntity<HttpStatus> deleteQuote(@PathVariable("id") Integer id) {
    try {
        quoteRepository.deleteById(id);
        return new ResponseEntity<>(HttpStatus.NO_CONTENT);
    } catch (RuntimeException e) {
        System.out.println(e.getMessage());
        return new ResponseEntity<>(HttpStatus.INTERNAL_SERVER_ERROR);
    }
}
}

```

新增資料庫設定

application.yaml

為服務存取的後端資料庫新增設定。編輯 (或建立，如果不存在) `src/main/resources` 下名為 `application.yaml` 的檔案，並為後端新增參數化 Spring 設定。


```
target: local

spring:
  config:
    activate:
      on-profile: cloud-dev
  datasource:
    url: 'jdbc:postgresql://${DB_HOST:127.0.0.1}/${DB_NAME:quote_db}'
    username: '${DB_USER:user}'
    password: '${DB_PASS:password}'
  jpa:
    properties:
      hibernate:
        jdbc:
          lob:
            non_contextual_creation: true
        dialect: org.hibernate.dialect.PostgreSQLDialect
    hibernate:
      ddl-auto: update
```

新增資料庫遷移

在 `src/main/resources/db/migration/` 建立資料夾

建立 SQL 檔案：`V1__create_quotes_table.sql`

將下列內容貼到檔案中

V1__create_quotes_table.sql

```
CREATE TABLE quotes(
  id INTEGER PRIMARY KEY,
  quote VARCHAR(1024),
  author VARCHAR(256)
);

INSERT INTO quotes (id,quote,author) VALUES (1,'Never, never, never give up','Winston Churchill');
INSERT INTO quotes (id,quote,author) VALUES (2,'While there''s life, there''s hope','Marcus Tullius Cicero');
INSERT INTO quotes (id,quote,author) VALUES (3,'Failure is success in progress','Anonymous');
INSERT INTO quotes (id,quote,author) VALUES (4,'Success demands singleness of purpose','Vincent Lombardi');
INSERT INTO quotes (id,quote,author) VALUES (5,'The shortest answer is doing','Lord Herbert');
```

Kubernetes 設定

在 `deployment.yaml` 檔案中加入下列內容，即可讓應用程式連線至 Cloud SQL 執行個體。

- TARGET - 設定變數，指出應用程式執行的環境

- `SPRING_PROFILES_ACTIVE` - 顯示作用中的 Spring 設定檔，該設定檔會設為 `cloud-dev`
- `DB_HOST` - 資料庫的私人 IP，建立資料庫執行個體時已記錄，或點選 Google Cloud 控制台導覽選單中的 `SQL` 即可查看 - 請變更值！
- `DB_USER` 和 `DB_PASS` - 如同在 CloudSQL 執行個體設定中設定，並以密鑰形式儲存在 GCP 中

使用下列內容更新 `deployment.yaml`。

`deployment.yaml`

```
apiVersion: v1
kind: Service
metadata:
  name: demo-app
  labels:
    app: demo-app
spec:
  ports:
    - port: 8080
      protocol: TCP
  clusterIP: None
  selector:
    app: demo-app
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: demo-app
  labels:
    app: demo-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: demo-app
  template:
    metadata:
      labels:
        app: demo-app
    spec:
      containers:
        - name: demo-app
          image: demo-app
          env:
            - name: PORT
              value: "8080"
            - name: TARGET
              value: "Local Dev - CloudSQL Database - K8s Cluster"
            - name: SPRING_PROFILES_ACTIVE
              value: cloud-dev
            - name: DB_HOST
              value: ${DB_INSTANCE_IP}
            - name: DB_PORT
              value: "5432"
            - name: DB_USER
              valueFrom:
                secretKeyRef:
                  name: gke-cloud-sql-secrets
                  key: username
            - name: DB_PASS
              valueFrom:
                secretKeyRef:
```

```
      name: gke-cloud-sql-secrets
      key: password
-   name: DB_NAME
    valueFrom:
      secretKeyRef:
        name: gke-cloud-sql-secrets
        key: database
```

將 DB_HOST 值替換為資料庫的位址

```
export DB_INSTANCE_IP=$(gcloud sql instances describe quote-db-instance \
  --format=json | jq \
  --raw-output ".ipAddresses[].ipAddress")

envsubst < deployment.yaml > deployment.new && mv deployment.new deployment.yaml
```

部署及驗證應用程式

1. 在 Cloud Shell 編輯器底部的窗格中，選取「Cloud Code」，然後選取畫面頂端的「Debug on Kubernetes」(在 Kubernetes 上偵錯)。
2. 建構和測試完成後，「輸出」分頁會顯示 `Resource deployment/demo-app status completed successfully`，並列出網址：「Forwarded URL from service demo-app: `http://localhost: 8080`」(從服務 demo-app 轉送的網址：`http://localhost:8080`)
3. 查看隨機名言

在 Cloud Shell 終端機中，針對 random-quote 端點多次執行下列指令。觀察重複呼叫傳回不同報價

```
curl -v 127.0.0.1:8080/random-quote
```

4. 新增報價

使用下列指令建立 ID 為 6 的新報價，並觀察要求是否回傳

```
curl -v -H 'Content-Type: application/json' -d '{"id":"6","author":"Henry David Thoreau","quote":"Go confidently in the direction of your dreams! Live the life you have imagined"}' -X POST 127.0.0.1:8080/quotes
```

5. 刪除報價

現在請使用刪除方法刪除剛才新增的引言，並觀察 `HTTP/1.1 204` 回應代碼。

```
curl -v -X DELETE 127.0.0.1:8080/quotes/6
```

6. 伺服器錯誤

在項目已刪除後再次執行最後的要求，就會發生錯誤狀態

```
curl -v -X DELETE 127.0.0.1:8080/quotes/6
```

請注意，回應會傳回 `HTTP:500 Internal Server Error`。

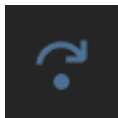
對應用程式進行偵錯

在上一節中，您嘗試刪除資料庫中沒有的項目時，應用程式發生錯誤。在本節中，您將設定中斷點來找出問題。錯誤發生在 DELETE 作業中，因此您會使用 `QuoteController` 類別。

1. 開啟 `src.main.java.com.example.springboot.QuoteController.java`
2. 找出 `deleteQuote()` 方法
3. 找出從資料庫刪除項目的程式碼行：`quoteRepository.deleteById(id);`
4. 點選行號左側的空白處，在該行設定中斷點。
5. 系統會顯示紅色指標，表示已設定中斷點
6. 再次執行 `delete` 指令

```
curl -v -X DELETE 127.0.0.1:8080/quotes/6
```

7. 按一下左欄中的圖示，切換回偵錯檢視畫面
8. 觀察在 `QuoteController` 類別中停止的偵錯行。
9. 在偵錯工具中，按一下 `step over` 圖示



，並觀察是否擲回例外狀況

10. 請注意，這會向用戶端傳回非常通用的「內部伺服器錯誤 HTTP 500」，這並非理想做法。`RuntimeException was caught.`

```
Trying 127.0.0.1:8080...
* Connected to 127.0.0.1 (127.0.0.1) port 8080 (#0)
> DELETE /quotes/6 HTTP/1.1
> Host: 127.0.0.1:8080
> User-Agent: curl/7.74.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 500
< Content-Length: 0
< Date:
<
* Connection #0 to host 127.0.0.1 left intact
```

更新程式碼

程式碼有誤，應重構例外狀況區塊，以擷取 `EmptyResultDataAccessException` 例外狀況，並傳回 HTTP 404 找不到狀態碼。

修正錯誤。

1. 在偵錯工作階段仍在執行的情況下，按下偵錯控制面板中的「繼續」按鈕，完成要求。
2. 接著，將下列區塊新增至程式碼：

```
    } catch (EmptyResultDataAccessException e){  
        return new ResponseEntity<HttpStatus>(HttpStatus.NOT_FOUND);  
    }  
}
```

方法應如下所示：

```
public ResponseEntity<HttpStatus> deleteQuote(@PathVariable("id") Integer id) {  
    try {  
        quoteRepository.deleteById(id);  
        return new ResponseEntity<>(HttpStatus.NO_CONTENT);  
    } catch (EmptyResultDataAccessException e){  
        return new ResponseEntity<HttpStatus>(HttpStatus.NOT_FOUND);  
    } catch (RuntimeException e) {  
        System.out.println(e.getMessage());  
        return new ResponseEntity<>(HttpStatus.INTERNAL_SERVER_ERROR);  
    }  
}
```

3. 重新執行刪除指令

```
curl -v -X DELETE 127.0.0.1:8080/quotes/6
```

4. 逐步執行偵錯工具，觀察 `EmptyResultDataAccessException` 是否遭到攔截，以及 HTTP 404 Not Found 是否傳回給呼叫端。

```
Trying 127.0.0.1:8080...  
* Connected to 127.0.0.1 (127.0.0.1) port 8080 (#0)  
> DELETE /quotes/6 HTTP/1.1  
> Host: 127.0.0.1:8080  
> User-Agent: curl/7.74.0  
> Accept: */*  
>  
* Mark bundle as not supporting multiuse  
< HTTP/1.1 404  
< Content-Length: 0  
< Date:  
<  
* Connection #0 to host 127.0.0.1 left intact
```

5. 按一下偵錯工具列中的紅色方塊



，停止偵錯工作階段。

6. 清除所用資源

恭喜！在本實驗室中，您從頭開始建立新的 Java 應用程式，並設定該應用程式，使其能有效與容器搭配運作。然後，您按照傳統應用程式堆疊中的相同開發人員流程，將應用程式部署至遠端 GKE 叢集並進行偵錯。

完成實驗室後，請執行下列清理作業：

1. 刪除實驗室中使用的檔案

```
cd ~ && rm -rf container-developer-workshop
```

2. 刪除專案，移除所有相關基礎架構和資源
-